CSC3002 | Mock solutions and marking guide

Open after attempting the question paper

This is a practice marking scheme. The user's confirmed exam structure is 20+40+40 marks; the individual weights and exact-set multiple-choice policy are mock design choices.

Quick key

Question	Answer	Points
Q1	B	5
Q2	C	5
Q3	D	5
Q4	A	5
Q5	A, B, C	6
Q6	A, B, D, F	6
Q7	A, B, E	6
Q8	B, C, D, E	6
Q9	A, B, E	6
Q10	A, C, E	5
Q11	B, D, E	5
Q12	Worked below	12
Q13	Worked below	14
Q14	Worked below	14

Score record

Part I: _____ /20 Part II: _____ /40 Part III: _____ /40

Total: _____ /100 Unanswered marks: _____ Guessed questions: _____

Part I | Single-choice explanations

Q1: B, 3 9 14. a is copied from x=3 and becomes 5. b aliases y and becomes 4+5=9. *c aliases z and becomes 5+9=14. x remains 3. The unchanged caller x does not undo the new local value of a.

Q2: C, 5 6.5 5.5. pick(4) has an exact int match. pick(4.5) uses the double overload with extra=2. pick(4.5,1) supplies extra=1. The first call is not ambiguous: the exact int match beats conversion to double.

Q3: D, 2 1 0 0. The first increment is skipped because true||... is already true. The second runs: visits becomes 2, and 2>2 is false. The third is skipped because b is false. Default stream formatting prints booleans as 1/0.

Q4: A. half(9) instantiates int arithmetic and returns int 4. half(9.0) returns double 4.5. c converts an already truncated int 4 into double 4.0. A double destination cannot recover a discarded fraction.

Mock marking policy

For each Part II question, award its stated points only for the complete correct set; otherwise zero. For learning, separately record each false option you selected and each true option you omitted. This avoids interpreting every zero as the same kind of misunderstanding. Follow the real paper's rules on exam day.

Part II | Arrays and strings

Q5. A, B, C [6]

A true: array parameters adjust to pointers; the bracketed bound is not a size check.

B true: one-past pointer formation is permitted; reading through it is not.

C true: read is a reseatable pointer to const int, while fixed is a fixed pointer to modifiable int.

D false: those are precisely the prohibited operations.

E false: sizeof(a) in inspect is sizeof(int*).

F false: a built-in array cannot be assigned a new pointer value.

Personal reminder: separate the pointer object, pointed-to object and original array. This question directly revisits your array-bound and a+4 follow-ups.

Q6. A, B, D, F [6]

A true: label contains G, o and '\0'; strlen counts the first two.

B true: substr returns a separate string; the second argument is a count.

C false: two string literals do not have a concatenation operator+.

D true: either operand may supply the std::string overload.

E false: label has only three slots; Blue plus its terminator requires five. strcpy cannot resize the array, so this copy would overflow.

F true: equal C-string contents produce zero from strcmp.

Personal reminder: do not confuse allocated bytes, content length and terminator storage.

Part II | Classes and dispatch

Q7. A, B, E [6]

A true: both constructors increment one shared variable.

B true: a const method is callable on both kinds of object.

C false: code has no in-class initializer, and assigning it in the constructor body cannot initialize the const member.

D false: created belongs to the class.

E true: trailing const constrains access through this; it does not require a const caller.

F false: static member functions have no this pointer.

Personal reminder: const concerns permitted modification; static concerns shared state or storage/linkage depending on context.

Q8. B, C, D, E [6]

A false: report remains pure virtual, so Reporter is abstract. A destructor does not remove this restriction.

B true: virtual dispatch reaches Brief::report.

C true: tag is nonvirtual and r has static type Reporter&, so Reporter::tag is chosen.

D true: missing const changes the member signature; override diagnoses that it does not override the base method.

E true: the virtual base destructor enables correct destruction through Reporter*.

F false: base private members are not directly accessible to derived-class code.

Personal reminder: the same function name is not enough for overriding. Check parameters and cv-qualification, then check whether the base operation is virtual.

Part II | Ownership, streams and iteration

Q9. A, B, E [6]

A true: allocation form determines deletion form; array length 1 does not change new[] into scalar new.

B true: assigning nullptr to p does not update q.

C false: weak_ptr adds no strong owner; s.use_count() is 1.

D false: unique_ptr is noncopyable; ownership can be moved.

E true: separately allocated rows need their own release; deleting their pointer array does not delete the rows.

F false: memberwise copying copies the raw pointer value, not the owned allocation.

Personal reminder: after every allocation/copy/delete, label each object as owner, observer, live or destroyed. Nullifying one variable does not repair all aliases.

Q10. A, C, E [5]

A true: the destinations are memory and file respectively.

B false: str() returns a string; output or storage requires another operation, and the getter does not clear the buffer.

C true: fixed with precision 2 writes two digits after the decimal point.

D false: clear resets state flags, not unread data.

E true: the read-controlled loop enters its body only when extraction succeeds.

Personal reminder: fixed is formatting state, not a destination. The stream receives the formatted characters.

Q11. B, D, E [5]

A false: deduction finds conflicting types int and double for one T.

B true: explicitly supplying double permits the int argument to convert.

C false: end is one past the last element.

D true: x is a copy.

E true: x is a reference to each vector element.

Personal reminder: auto versus auto& changes mutation behavior; explicit template arguments and deduced arguments are not identical processes.

Q12 | Trace, write and diagnose

12 points

(a) Pointer trace [6]

```
6 6 10 10
2 11
```

Assignment	Pointer index afterward	Assigned result	Array change
a=*++p	1	a=6	None
b=*p++	2	b=6	None
c=++*p	2	c=10	data[2]: 9 to 10
d=(*p)++	2	d=10	data[2]: 10 to 11

Final array: {3,6,11,12,15}. The assignments to a and b move p. The assignments to c and d modify its pointee. p-data is an element distance, so it is 2; *p is 11.

Rubric: first line 2 points (0.5 per value); second line 2 (1 per value); final array 1; correct pointer-moving assignments 1.

Q12 | Callback and lifetime

(b) Callback [4]

```
bool isEven(long value) {
    return value % 2 == 0;
}
int countIf(const long* a, int n, bool (*pred)(long)) {
    int count=0;
    for(int i=0; i<n; ++i) {
        if(pred(a[i])) ++count;
    }
    return count;
}
bool (*test)(long)=isEven;
// long data[]={3,4,8,9};
// countIf(data,4,test) returns 2.
```

`(*pred)(a[i])` is also a valid call. The function-pointer parameter represents behavior supplied by the caller. countIf does not need to know what the predicate tests.

Rubric: correct predicate 1; safe loop invoking pred and returning the count 2; explicit function-pointer declaration plus result 2 receives 1 total (0.5 each). Equivalent correct implementations earn credit, including one handling n=0 without reading the array.

(c) Lifetime [2]

The function can compile, often with a warning. local is an automatic object whose lifetime ends when bad returns. q consequently dangles; reading *q has undefined behavior. There is no guaranteed output of 7.

Rubric: correct classification 1; explanation of local lifetime/dangling pointer 1. Returning an int by value would be a simple repair when only the value is needed; allocating dynamically would require an explicit ownership plan.

Q13 | Operators and abstract interfaces

14 points

(a) A valid implementation [7]

```
struct Money {
    int cents;
    Money operator+(const Money& rhs) const {
        return Money{cents + rhs.cents};
    }
    Money& operator+=(const Money& rhs) {
        cents += rhs.cents;
        return *this;
    }
};
```

Rubric for + [4]: appropriate by-value return and member signature 1; const-reference right operand 1; trailing const 1; correct new sum without modifying inputs 1.

Rubric for += [3]: suitable mutable member signature returning Money& and accepting the right operand 1; correct mutation 1; return *this 1. This return supports (a+=b)+=c. An equivalent operator+ using a local copy and += is valid if it preserves the same const-correct contract.

Do not return Money& referring to a local result; that reference would dangle. Do not assume += updates both objects: rhs is read-only through its const reference.

(b) Const [2]

const Money& rhs provides read-only access to the right operand through rhs and avoids a parameter copy [1]. Trailing const restricts changes to the left object's ordinary state through this and allows const left operands [0.5]. += changes cents, so it cannot promise that trailing-const contract for this class [0.5].

Q13 | Abstract interfaces and dispatch

(c) Abstract interface [3]

```
struct SilentReporter : Reporter {
    int report() const override { return 0; }
};
```

A shared interface allows client code to call report without depending on a particular reporter implementation [1]. Reporter cannot be instantiated because report is pure virtual [1]. SilentReporter provides the matching override, including const, and is concrete [1]. A class inheriting an already concrete override could also be concrete without repeating it.

(d) Virtual dispatch [2]

The output is `0` [1]. Although r has static type Reporter&, it refers to a SilentReporter. report is virtual, so the final override for the dynamic object type is selected [1]. The virtual destructor supports safe ordinary deletion through a base pointer; no dynamic allocation is needed for this particular reference call.

Optional self-check after marking

With Money a{120}, b{30}, c=a+b, a remains 120, b remains 30 and c is 150. After (a+=b)+=b, a is 180. This is a useful contrast between producing a new value and mutating an existing one.

Q14 | Templates and streams

14 points

(a) Template [4] and calls [4]

```
template<typename T>
T halfValue(T value) {
    return value/2;
}
```

Template rubric: template type-parameter declaration 1; return type T 1; parameter T value 1; correct return expression 1.

Variable	Type	Value	Why
a	int	3	Deduced int division
b	double	3.5	Deduced double division
c	double	3.0	Truncated int return converted afterward
d	double	3.5	Explicit T=double converts the argument before division

Award 1 point per row, 0.5 each for type and value. auto follows the call's result type; it does not mean double.

(b) Output and reasoning [4]

```
24:[]
[Ada Lovelace]
__3.50|2.00
```

`>> id` consumes 24 but leaves the newline. The first getline consumes that newline and returns an empty string. The second obtains the full name including its internal space. The formatted value 3.50 occupies four characters; width 6 adds two underscores. Width resets after this field; fixed and precision remain, so 2.0 becomes 2.00. Fill also persists, although no later width requests padding. out.str() supplies the built string to cout.

Rubric: first line 1; name line 1; formatted line 1; pending-newline explanation and correct persistence explanation 1 total (0.5 each).

Q14 | Input recovery and revision

(c) Recovery [2]

The loop stops with total=30. A valid continuation for this exact input is:

```
numbers.clear();
char bad;
numbers >> bad;       // consumes x, with leading whitespace skipped
if(numbers >> n) total+=n;
// total is now 60
```

clear() resets fail flags but does not remove x. Without consuming that character, integer extraction encounters x again and fails again. The question guarantees one bad character followed by 30; arbitrary malformed input may need a different recovery policy.

Rubric: total before recovery 0.5; correct clear-and-consume sequence 0.5; guarded read/add 0.5; explanation of clear alone 0.5.

What to redo after grading

Missed Q1/Q5/Q12: rebuild a pointer/alias state table. Missed Q3/Q4/Q14(a): annotate skipped operations and types before calculating. Missed Q7/Q8/Q13: write the class signatures and explain each qualifier. Missed Q9: draw owners and destroyed allocations. Missed Q10/Q14(b-c): write the remaining input and current stream flags/settings after every statement. Redo a changed-input version before calling a topic mastered.

Coverage and source map

What the mock sampled, and what remains

- Q1-Q4: caller state, overload/default rules, short-circuiting and integer-vs-floating arithmetic.

- Q5-Q8: arrays, const pointers, strings, static/const members, abstract classes and dispatch.

- Q9-Q11: lifetime/ownership, stream semantics and iterator/template rules.

- Q12-Q14: writing function-pointer code, pointer tracing, lifetime diagnosis, writing operators/interfaces/templates, and explaining stream state.

The paper does not fully sample L3 interfaces/enums/randomness, generic selection-sort implementation, full class-template definitions, complete Rule-of-Three implementations, matrix code, advanced string-processing loops, Rational normalization or TokenScanner usage. These remain on the review checklist. Their absence is a practice-paper length choice, not an exam exclusion.

Evidence map

Main tutorial slide ranges: Q1/2/3/12(b) -> slides 4-11; Q5/6/12(a) -> slides 12-16; Q7/8/13 -> slides 17-22; Q4/11/14(a) -> slides 23-25 plus L9. Q9, Q10, Q12(c), Q14(b-c) additionally target L5/L7 and your prior feedback. The companion review records filenames and missing-photo limitations.